Linux Network Configuration Generator — User Guide

Table of Contents

1. Quick Start

Both a **systemd-networkd** config and an **iproute2 shell-script** config are generated from the same single command:

```
python3 -m src.main \
    --input input/examples/configuration_end-station_Linux.json \
    --os    linux
```

Expected output:

```
YANG validation passed (interfaces, routes, DNS)
Config generation completed for plugin: linux
```

The results are written to `output/linux/`:

```
output/linux/
 ├──── overlay.sha256                              (1)
```

```
└──── overlay/
    ├──── etc/systemd/network/                    (2)
    │     ├──── 10-eth1.link
    │     ├──── 20-eth1.network
    │     ├──── 30-eth1.2049.netdev
    │     ├──── 31-eth1.10.netdev
    │     ├──── 32-eth1.201.netdev
    │     ├──── 40-eth1.2049.network
    │     ├──── 41-eth1.10.network
    │     └──── 42-eth1.201.network
    └──── usr/lib/
        ├──── network-config/ip-config.sh         (3)
        └──── systemd/system/network-config.service
```

(1) SHA-256 integrity manifest.
(2) systemd-networkd unit files.
(3) iproute2 shell script + companion systemd service unit.

2. systemd-networkd Configuration

2.1. Can I build systemd-networkd configuration?

Yes. The generator produces the following unit files placed in the overlay under `etc/systemd/network/`:

File	Type	Purpose
`10-<iface>.link`	`.link`	Renames the kernel NIC to the IETF-configured name and sets MTU. Processed by udev before networkd starts.
`20-<iface>.network`	`.network`	Configures MAC address, MTU, and lists all VLAN sub-interfaces.
`3N-<vlan>.netdev`	`.netdev`	Creates one VLAN virtual device per 802.1Q sub-interface.
`4N-<vlan>.network`	`.network`	Assigns IPv4 address and static routes to each VLAN sub-interface.

2.2. Generated file contents

`10-eth1.link` — *NIC rename*

```
[Match]
OriginalName=pfe0
Type=ether

[Link]
```

```
Name=eth1
MTUBytes=1500
```

`OriginalName=pfe0` is the platform-specific kernel NIC name. Adapt it in `src/plugins/linux/systemd-network-config/templates/parent_interface_name_static.j2` for your hardware.

`20-eth1.network` — *parent interface*

```
[Match]
Name=eth1

[Link]
MACAddress=E4:D3:AA:96:FF:01
MTUBytes=1500

[Network]
VLAN=eth1.2049
VLAN=eth1.10
VLAN=eth1.201
```

`30-eth1.2049.netdev` — *VLAN virtual device*

```
[NetDev]
Name=eth1.2049
Kind=vlan

[VLAN]
Id=2049
```

`41-eth1.10.network` — *VLAN interface with IP and route*

```
[Match]
Name=eth1.10

[Network]
Address=172.16.10.11/24

[Link]
MTUBytes=1500

[Route]
Gateway=172.16.10.254
Destination=0.0.0.0/0
```

2.3. Deploy to target

```
# Copy unit files and restart networkd
rsync -a output/linux/overlay/etc/ /etc/
systemctl restart systemd-networkd
```

Or mount the full overlay via OverlayFS on a read-only root:

```
mount -t overlay overlay \
  -o lowerdir=/,upperdir=output/linux/overlay,workdir=/tmp/ov-work \
  /mnt/target
```

3. iproute2 / `ip` Configuration

3.1. Can I build an ip-based network configuration?

Yes. The generator produces `ip-config.sh` — a portable POSIX shell script using `ip(8)`. It is suited for targets without `systemd-networkd`: containers, custom init systems, or stripped-down images.

3.2. Generated `ip-config.sh` (excerpt)

```
#!/bin/sh
set -eu

# Override the physical parent interface name for QEMU / CI environments.
VLAN_PARENT="${VLAN_PARENT:-}"

# Phase 1 - physical interface
if [ "eth1" != "eth0" ] && ip link show eth0 >/dev/null 2>&1; then
    ip link set dev eth0 down
    ip link set dev eth0 name eth1
fi
ip link set dev eth1 address E4:D3:AA:96:FF:01
ip link set dev eth1 mtu 1500 up

# Phase 2 - VLAN sub-interfaces (each step is idempotent)
_parent="${VLAN_PARENT:-eth1}"
if ! ip link show eth1.2049 >/dev/null 2>&1; then
    ip link add link "${_parent}" name eth1.2049 type vlan id 2049
fi
ip link set dev eth1.2049 mtu 1500 up
if ! ip -4 addr show dev eth1.2049 | grep -q "inet 172.24.1.11/24"; then
    ip addr add 172.24.1.11/24 dev eth1.2049
fi
```

```
# Phase 3 - static routes
ip route replace default        via 172.16.10.254
ip route replace 172.24.0.0/20 via 172.24.1.254
```

3.3. Companion systemd service

```
# usr/lib/systemd/system/network-config.service
[Unit]
Description=IP Network Configuration
After=network-pre.target
Before=network.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/lib/network-config/ip-config.sh

[Install]
WantedBy=multi-user.target
```

3.4. Deploy to target

```
# Install script and service unit
install -m 755 output/linux/overlay/usr/lib/network-config/ip-config.sh \
               /usr/lib/network-config/ip-config.sh
install -m 644 output/linux/overlay/usr/lib/systemd/system/network-config.service \
               /usr/lib/systemd/system/network-config.service

# Enable and run
systemctl enable --now network-config.service

# Or run directly without systemd
sh /usr/lib/network-config/ip-config.sh
```

3.5. QEMU / CI override

Set `VLAN_PARENT` to redirect VLAN creation to a different interface:

```
VLAN_PARENT=eth0 sh output/linux/overlay/usr/lib/network-config/ip-config.sh
```

4. Input File

The example input is `input/examples/configuration_end-station_Linux.json`.

It defines:

- Physical interface `eth1` (MAC `E4:D3:AA:96:FF:01`, MTU 1500)
- VLAN sub-interfaces `eth1.2049` (172.24.1.11/24), `eth1.10` (172.16.10.11/24), `eth1.201` (172.16.201.11/24)
- Static routes: default via 172.16.10.254, 172.24.0.0/20 via 172.24.1.254
- Hostname: `FZCU_MPU`

Pass a different file via `--input`. The file must follow the IETF YANG data model (`ietf-interfaces`, `ietf-routing`, `ietf-system`).

5. Running the Tests

```
# All linux-specific tests (74 tests)
python3 -m pytest tests/linux/ -v

# All project tests (97 tests)
python3 -m pytest tests/ tests/linux/

# Individual modules
python3 -m pytest tests/linux/test_systemd_templates.py -v
python3 -m pytest tests/linux/test_ip_templates.py      -v
python3 -m pytest tests/linux/test_output_validator.py  -v
python3 -m pytest tests/linux/test_generator.py         -v
```

Expected result:

```
74 passed in X.XXs
```

6. Output Validation

Validation runs automatically after generation. To run it manually:

```
python3 src/plugins/linux/output_validator.py output/linux/overlay
# All checks passed.
```

Verify integrity on the target after deployment:

```
sha256sum -c output/linux/overlay.sha256
```

7. CLI Reference

```
python3 -m src.main [OPTIONS]

  --input   PATH   YANG JSON input file                   [required]
  --output  DIR    Output directory                       [default: output]
  --os      TEXT   Plugin: linux|android|mcu|qnx|switch  [required]
```

Standalone generator module (direct invocation, no core parse pipeline):

```
python3 -m src.plugins.linux.generator \
    --input  input/examples/configuration_end-station_Linux.json \
    --output output/linux \
    [--skip-yang-validation] \
    [--no-manifest]
```